AI • COMPUTER SCIENCE

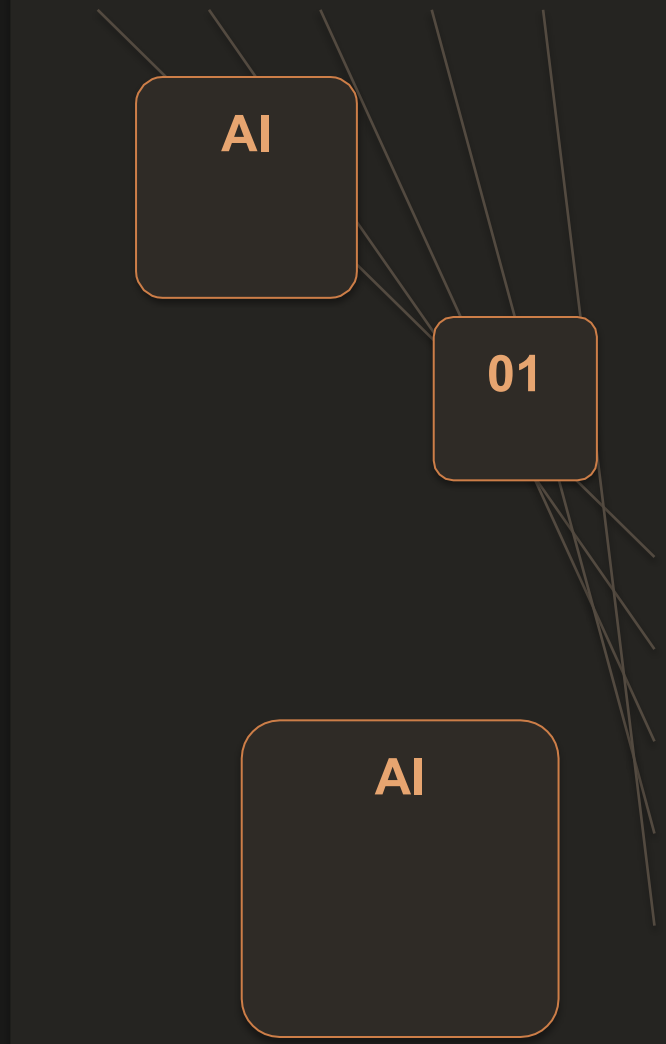
PROMPT ENGINEERING

Beyond asking AI questions.

How to communicate with AI systems to produce useful, relevant and reliable results.

A practical learning session for technology students

LEARN • APPLY • EVALUATE



01 / START HERE

The same AI. Different results.

Why can one prompt produce a useful answer while another produces a vague one?

LESS DIRECTION

Prompt A — Vague

“Build me a website.”

The AI must guess the audience, features, design, technology and expected result.

CLEARER DIRECTION

Prompt B — Specific

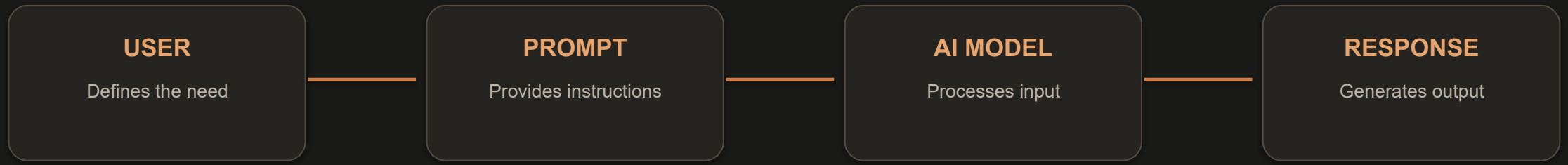
“Build a responsive portfolio for an AI student. Include projects, skills and contact sections. Use a clean dark theme.”

Key idea: better instructions help AI understand what success should look like.

What is Prompt Engineering?

A practical skill for working effectively with generative AI and large language models.

Prompt engineering is the process of designing, structuring and refining instructions given to an AI model to obtain useful, relevant and well-directed responses.



A prompt is the input. Prompt engineering improves the way that input is designed.

Why does it matter?

Prompting helps guide AI toward outputs that better match the task.

BENEFIT 01

ACCURACY

Reduces ambiguity and focuses the task.

BENEFIT 02

RELEVANCE

Adds audience and background context.

BENEFIT 03

EFFICIENCY

Reduces avoidable follow-up instructions.

BENEFIT 04

CONTROL

Specifies tone, format, scope and constraints.

BENEFIT 05

PRODUCTIVITY

Supports coding, research and analysis.

BENEFIT 06

CONSISTENCY

Makes repeated outputs easier to standardize.

A good prompt improves the process; it does not guarantee every AI answer is correct.

Anatomy of a strong prompt

Five useful building blocks. Use only the elements your task actually needs.

01	**ROLE**	Who should the AI act as?	*Act as a Python instructor.*
02	**TASK**	What should it do?	*Explain Python functions.*
03	**CONTEXT**	What background matters?	*For a beginner learning programming.*
04	**CONSTRAINTS**	What rules must it follow?	*Use simple language and one example.*
05	**OUTPUT**	How should it respond?	*Use headings and a short summary.*

ROLE + TASK + CONTEXT + CONSTRAINTS + OUTPUT FORMAT

Basic prompting techniques

The number of examples you provide changes how much guidance the model receives.

NO EXAMPLES

ZERO-SHOT

Give an instruction without examples.

“Explain the difference between AI and machine learning.”

ONE EXAMPLE

ONE-SHOT

Give one example to show the expected pattern.

“happy → Positive. Classify: excellent.”

MULTIPLE EXAMPLES

FEW-SHOT

Give multiple examples to demonstrate a pattern.

“great → Positive; awful → Negative. Classify: useful.”

Use zero-shot for clear tasks; use examples when format or pattern matters.

More techniques worth knowing

These techniques help guide complex tasks and produce more predictable outputs.

TECHNIQUE 01

ROLE-BASED

Assign a useful perspective or expertise.

TECHNIQUE 02

CONTEXTUAL

Provide relevant background information.

TECHNIQUE 03

STRUCTURED OUTPUT

Request JSON, tables, lists or a schema.

TECHNIQUE 04

ITERATIVE

Refine the response through follow-up prompts.

TECHNIQUE 05

TASK DECOMPOSITION

Break a large task into smaller steps.

TECHNIQUE 06

RAG PROMPTING

Ground answers in retrieved or supplied information.

Combine techniques thoughtfully instead of making every prompt unnecessarily long.

Prompting in software development

Use prompts to assist development—not to skip testing or engineering judgment.

EXAMPLE PROMPT

Act as a senior Python developer.

Build a basic student-record REST API.

Include:

- Create, read, update and delete operations
- Input validation and error handling
- Clear project structure
- Example requests and test cases

Explain how to run and test the project.

BUILD

CODE GENERATION

Create starter code and explain its structure.

IMPROVE

DEBUGGING

Explain an error and suggest a fix to verify.

Review generated code, test edge cases and understand what you deploy.

SEO & cybersecurity prompting

Two examples of precise instructions in different technical domains.

SEO — SEARCH ENGINE OPTIMIZATION

Act as an SEO content specialist.

Suggest relevant keywords and a heading structure for an article about beginner Python programming.

Consider search intent, readability and useful metadata. Avoid keyword stuffing.

DEFENSIVE CYBERSECURITY

Act as a defensive application security reviewer.

Review this code for input validation, SQL injection risks and insecure authentication.

Suggest safe fixes. Do not perform attacks.

SEO improves discoverability. Security prompts should support authorized, defensive work.

One master prompt for a website

Complex projects need context, functionality, design rules, technical choices and verification.

WEBSITE PROJECT PROMPT

Act as a senior frontend developer and UI/UX designer.

Build a responsive portfolio website for an AI student.
Include Home, About, Skills, Projects and Contact sections.

Requirements:
- Professional, accessible interface
- Responsive layout for mobile and desktop
- Working navigation and buttons
- SEO-friendly page structure
- Secure form validation
- Clear setup and deployment instructions

First inspect the project files. Explain your plan,
implement the website and provide testing steps.

STRUCTURE

WHY IT WORKS

Defines the role, goal, audience, features and expected output.

VERIFY

WHAT TO CHECK

Test navigation, responsiveness, forms, accessibility and security.

A master prompt is useful for a complex project—not necessary for every simple question.

Common mistakes & how to evaluate output

Prompting is iterative: write, inspect, test and refine.

AVOID

COMMON MISTAKES

- Vague instructions
- Missing context
- Conflicting constraints
- Too many unrelated tasks
- Trusting unverified output

EVALUATE

QUALITY CHECKLIST

- Does it answer the actual task?
- Is the information accurate?
- Does it follow the requested format?
- Are assumptions stated?
- Can the result be tested or verified?

AI can make mistakes or hallucinate. Verify important claims and generated code.

KEY TAKEAWAYS

Prompting is a skill.

The goal is not to write the longest prompt. The goal is to communicate the task clearly.

- Start simple; add detail when the task needs it.
- Give relevant context and clear constraints.
- Use examples and structure when consistency matters.
- Break complex tasks into manageable stages.
- Evaluate AI output instead of accepting it blindly.

LEARN • EXPERIMENT • EVALUATE • IMPROVE